

RELEVAMIENTO FINAL · TRASPASO A DESARROLLO

Home Pádel: dónde está el proyecto y qué falta

El núcleo —seguridad, pagos, precios, stock y variantes— está cerrado y verificado contra el servidor. Quedan dos funcionalidades a medias que hoy le cuestan plata al negocio, una prueba pendiente y tres trámites del dueño de las cuentas.

FECHA	COMMIT	STACK
6 de septiembre de 2026	ffb4b90 · main	NestJS · Next.js 15 · Prisma · PostgreSQL
INFRA		
Vercel (web) · Railway (API y base)		

0 credenciales expuestas en producción	43 pruebas automatizadas, todas pasando	2 funcionalidades a medias que cuestan plata	1 prueba de pago nunca ejecutada
---	--	---	-------------------------------------

Cómo levantar el proyecto

Tres aplicaciones en un mismo repositorio. Cada una tiene su `.env.example` con las variables explicadas y cuáles son obligatorias.

```
backend/      API REST — NestJS + Prisma + PostgreSQL    → puerto 4000
frontend/     Tienda pública — Next.js 15 (App Router)         → puerto 3000
backoffice/   Panel de administración — Next.js 15             → puerto 3001

# En cada carpeta
npm install
cp .env.example .env          # .env.local en frontend y backoffice
npm run dev

# Solo en backend, la primera vez
npx prisma generate
npx prisma migrate deploy

# Pruebas (backend)
npm test
```

Qué está resuelto

Verificado contra el servidor en producción, no contra los mensajes de los commits.

ÁREA	ESTADO	CÓMO SE COMPROBÓ
Credenciales fuera del endpoint público	CERRADO	cloudinary y payment_methods sin secretos; publicKey conservada
Firma del aviso de pago de Mercado Pago	CERRADO	manifest correcto; 13 casos cubiertos por pruebas
Suplantación por login con Google	CERRADO	ruta eliminada · HTTP 404
Precio calculado en el servidor	CERRADO	se descarta el precio que envía el navegador
Stock descontado de forma atómica	CERRADO	condición dentro del UPDATE, por variante o producto
Variantes de talle y color	CERRADO	ficha → carrito → orden → stock; 39 productos cargados
Imágenes en Cloudinary	CERRADO	sin base64; ya no se pierden al desplegar
Consumo de transferencia en Vercel	CERRADO	regeneración cada hora en lugar de cada minuto
Límite de peticiones	CERRADO	ThrottlerGuard registrado globalmente
Despliegue y migraciones	CERRADO	migrate deploy; sin borrado de datos al arrancar
Endpoints privados protegidos	CERRADO	401 sin credenciales en users, orders, expenses
Variables de entorno documentadas	CERRADO	.env.example en las tres aplicaciones

Prioridad 1 — funcionalidades a medias

Las dos se ven terminadas desde la pantalla y no lo están. Por eso conviene no darlas por buenas mirando la interfaz: hay que seguir el dato hasta la orden.

P1 COBRA DE MENOS

El costo de envío se muestra al comprador pero no se cobra

```
carrito/page.tsx:17      subtotal >= 100000 ? 0 : 4500
checkout/page.tsx:47     subtotal >= 100000 ? 0 : 4500
```

El monto y el umbral de envío gratis están escritos a mano en dos archivos, sin pantalla en el backoffice para cambiarlos.

EL PROBLEMA REAL

orderData no incluye shipping y la preferencia de Mercado Pago se arma solo con los productos. El comprador ve «Envío \$ 4.500», Mercado Pago le cobra únicamente la mercadería y la orden queda con envío en cero. La tienda regala el envío en cada venta.

CRITERIO DE ACEPTACIÓN

```
Tarifa y umbral configurables desde el backoffice y calculados en el
servidor. El envío viaja como ítem en la preferencia de Mercado Pago y
queda guardado en la orden. Sin literales en el frontend:
grep -rn "4500" frontend/src/app/carrito frontend/src/app/checkout → 0
```

P2 PROMETE DE MÁS**Los cupones son una simulación en el navegador**

```
carrito/page.tsx:22
const VALID_COUPONS = ['HOMEPADEL10', 'PRIMERA10', 'DESCUENTO10'];
const discount = couponApplied ? Math.round(subtotal * 0.1) : 0;
```

Tres códigos y un 10% fijo, en el código que se descarga al navegador: cualquiera que mire el fuente los ve.

LO LLAMATIVO

La infraestructura real existe y no se usa: modelo **Coupon** con monto mínimo, usos máximos y vencimiento; pantalla de cupones en el backoffice; endpoints de validación en el backend. Nada conectado al carrito.

EL PROBLEMA REAL

El descuento no viaja al servidor. El comprador ve «Cupón aplicado (10% OFF)» y termina pagando el total sin descuento.

CRITERIO DE ACEPTACIÓN

Un cupón creado en el backoffice se aplica; uno inexistente o vencido se rechaza. El descuento aparece en el total cobrado y queda en la orden.

```
grep -rn "VALID_COUPONS" frontend/src → 0
```

Prioridad 2 — la prueba que falta**P3 SIN EJECUTAR****El circuito de cobro nunca se probó de punta a punta**

Es lo único de todo el proyecto que no pudo verificarse desde el código. La firma del aviso de pago está implementada según la documentación de Mercado Pago y cubierta por pruebas, pero que Mercado Pago la acepte en vivo solo se confirma pagando.

REQUISITO PREVIO

Las tres variables cargadas en Railway (ver O2). Sin

MERCADOPAGO_WEBHOOK_SECRET el aviso se rechaza por diseño y la venta no se registra.

CRITERIO DE ACEPTACIÓN

Un pago de prueba aprobado que deje:

- la orden en estado **PAID** en el backoffice
- el stock de la variante **descontado**
- el comprador en **/checkout/success** con su número de orden

Prioridad 3 — deuda técnica

No rompe nada hoy. Conviene resolverlo cuando se toque cada zona.

Errores descartados en silencio

- Quedan **15** bloques `catch {}` vacíos en pantallas de configuración del backoffice.
- Se corrigieron los cuatro de mayor uso: subida de imágenes en productos, categorías y marcas, y la carga del listado.
- Es el origen de los reportes del tipo «no me deja y no sé por qué».

Cobertura de pruebas

- Hay **43** pruebas sobre los tres caminos críticos: precio y stock, firma de pago, saneamiento de credenciales.
- Sin cobertura: órdenes, cupones, envíos, subidas y todo el frontend.
- Al tocar P1 o P2 conviene sumar pruebas en el mismo movimiento.

Integración continua

- No hay ningún flujo automático que corra `npm test` ni la compilación antes de fusionar.
- Con las pruebas ya escritas, agregarlo es media hora y evita que una regresión llegue a producción.

Envíos con Correo Argentino

- El módulo existe pero está vacío: `// TODO`.
- Depende de credenciales de MiCorreo que todavía no se gestionaron (ver O3).
- Relacionado con P1: conviene definir primero cómo se calcula el envío.

Depende del dueño de las cuentas

No son tareas de programación: requieren paneles a los que el equipo de desarrollo no tiene acceso.

01 SEGURIDAD

Rotar las claves de Cloudinary

El `apiSecret` estuvo accesible sin autenticación mientras la corrección no estaba desplegada. Hoy el endpoint ya no lo expone, pero eso no deshace el período en que estuvo público: hay que generar credenciales nuevas y reemplazarlas en Railway.

02 CONFIGURACIÓN

Confirmar tres variables en Railway

`FRONTEND_URL`, `BACKEND_URL` y `MERCADOPAGO_WEBHOOK_SECRET`. No pueden verificarse desde afuera. Sin las dos primeras el checkout falla de forma visible; sin la tercera se rechazan los avisos de pago y las ventas no se registran.

03 INTEGRACIONES

Tres integraciones pendientes de trámite

Resend — verificar el dominio para que los correos de la tienda no caigan en spam. Requiere registros DNS.

Correo Argentino — registrarse en MiCorreo con el CUIT del negocio y pedir las credenciales. Es el trámite más lento.

Instagram — desde junio de 2026 Meta no exige token ni revisión para incrustar publicaciones públicas. Conviene probar si funciona sin configurar nada antes de asumir que hay trabajo.

Verificación rápida del estado

Para comprobar la salud del sistema sin depender de este documento.

```
API=https://homepadeL-production.up.railway.app/api

# Sin credenciales expuestas — ambos deben dar 0
curl -s $API/site-sections/cloudinary | grep -c apiSecret
curl -s $API/site-sections/payment_methods | grep -c accessToken

# Sin imágenes pesadas embebidas — debe dar 0
curl -s $API/hero-slides | grep -c 'data:image'

# Endpoints privados protegidos — los tres deben dar 401
for e in users orders expenses; do
  echo "$e: $(curl -s -o /dev/null -w '%{http_code}' $API/$e)"
done

# Ningún producto a precio cero
curl -s "$API/products?limit=100" | grep -c '"salePrice":0'

# Pruebas y compilación
cd backend    && npm test && npx tsc --noEmit
cd backoffice && npx tsc --noEmit
cd frontend   && npx next build
```

Relevamiento sobre el commit `ffb4b90` de `main`. Las cifras de credenciales, imágenes, precios y protección de endpoints se midieron directamente contra el servidor de producción, no se estimaron. Sin cobertura: pruebas manuales de cada pantalla del backoffice, el circuito de pago ejecutado de verdad (P3), accesibilidad, comportamiento en móviles y rendimiento del frontend más allá del peso de las respuestas.